

Stage 3 — File Ingestion

Sprint: Sprint 3

Status: COMPLETE

Goal

Expand the Personal AI Agent so that knowledge can be loaded directly from files instead of requiring note text to be manually supplied inside Python code.

The target workflow for this stage was:

File Path → File Type Detection → Text Extraction → Note Identification → Duplicate Check → Chunking → Embeddings → Vector Storage

The supported file types introduced during this stage are:

- Plain-text files (.txt)
- PDF documents (.pdf)

The existing RAG pipeline from Stage 2 remains responsible for chunking, embedding, storage, retrieval, prompt construction, and answer generation.

Stage 3 adds the file-processing layer that prepares real documents for that pipeline.

Starting Point

At the end of Stage 2, the system had a complete working RAG pipeline.

The system could:

- Accept note text supplied directly to `ingest_note()`.
- Divide the text into chunks.
- Generate embeddings through `EmbeddingGemma`.
- Store the chunks and their metadata in `ChromaDB`.
- Retrieve semantically related chunks.
- Build a prompt from the retrieved text.
- Generate a grounded answer using `Qwen3 4B`.

However, the system did not yet have a general method for reading knowledge directly from files.

The ingestion process effectively started with:

Python String → ingest_note() → Chunking → Embeddings → ChromaDB

Stage 3 extended the beginning of that process:

User File → Extracted String → ingest_note() → Chunking → Embeddings → ChromaDB

Text File Loading

Text-file loading was implemented under:

backend/app/ingestion/file_loader.py

A function was created:

```
load_txt_file(file_path)
```

The function receives the path to a .txt file.

Its workflow is:

Text File Path → Open File in Read Mode → Read UTF-8 Text → Return String

The file is opened using a Python with block.

This ensures that the file is closed automatically after its contents have been read.

UTF-8 encoding is specified so that common letters, symbols, and other text characters can be interpreted correctly.

The function returns the complete contents of the file as a Python string.

Example conceptual result:

data/Apple_china.txt → load_txt_file() → Extracted Text String

The returned string can then be passed into the existing ingestion pipeline.

File Path Resolution

Testing exposed an important distinction between:

- The location of the Python file

- The current working directory used to execute Python

Relative file paths are normally interpreted from the current working directory, not automatically from the location of the Python script.

The project structure places the application and local data under the backend directory:

backend/app/

backend/data/

The VS Code workspace configuration was adjusted so that Python execution could consistently recognize the backend directory and the app package.

A .vscode/settings.json configuration was added at the project workspace level.

The configuration established:

- backend as the terminal working directory.
- backend as part of Python's module search path.
- Consistent imports beginning with app.
- Execution through the selected project virtual environment.

This allows project imports such as:

```
from app.ingestion.file_loader import load_txt_file
```

and file paths such as:

```
data/Apple_china.txt
```

to work consistently when files are executed through VS Code.

Text File Ingestion

The text loader was connected to the existing ingestion pipeline.

The completed text-file workflow is:

TXT File Path

→ load_txt_file()

→ Extracted Text String

→ Generate Note Information

→ Check for Existing Note

→ ingest_note()

→ chunk_text()

→ EmbeddingGemma

→ Vector Storage

This means the application no longer requires the complete note text to be manually written inside the test or ingestion code.

The user only needs to provide the file path.

Dynamic Note Identification

The Python `pathlib.Path` class was used to extract file information automatically.

For a file path such as:

`data/Apple_china.txt`

the following properties are available:

`file.stem`

Returns:

`Apple_china`

`file.name`

Returns:

`Apple_china.txt`

`file.suffix`

Returns:

`.txt`

The filename without its extension is used as the note ID.

The complete filename is used as the source.

Example:

File Path: data/Apple_china.txt

Note ID: Apple_china

Source: Apple_china.txt

The same approach also works for PDF files.

Example:

File Path: ``://data/Neurogum.pdf ****Note ID:****Neurogum ****Source:****Neurogum.pdf`

This removes the need to manually enter a separate note ID and source for every file.

Duplicate-Ingestion Protection

A duplicate check was added before a file is ingested.

The system searches the collection metadata for the dynamically generated note_id.

The workflow is:

Generate Note ID → Search Existing Metadata → Decide Whether to Ingest

If the note ID already exists, the system skips the ingestion operation.

This prevents the same file from being unintentionally processed repeatedly.

The distinction between Chroma record IDs and note IDs remains important.

A single note may produce several chunk records:

- Neurogum_chunk_0
- Neurogum_chunk_1
- Neurogum_chunk_2

Each chunk has a unique Chroma record ID.

All chunks from the same file also contain:

note_id: Neurogum

in their metadata.

ChromaDB prevents duplicate record IDs when records are added using the same IDs. The application-level duplicate check improves this behavior by detecting the existing note before ChromaDB raises a duplicate-ID error.

The resulting behavior is clearer:

Existing Note → Skip Ingestion → Display Informative Message

File Validation

File validation was added to prevent invalid files from silently entering the ingestion workflow.

The validation process checks:

- Whether the supplied file exists.
- Whether its extension is supported.
- Whether the requested loader matches the file type.

This allows the application to produce a clear error for:

- An incorrect file path
- A missing file
- An unsupported file type

The currently supported extensions are:

- .txt
- .pdf

Other file types are rejected instead of being interpreted incorrectly.

General File Routing

A general file-ingestion entry point was implemented.

The application examines the file extension and selects the correct loader.

The workflow is:

File Path → Inspect File Suffix

For .txt:

load_txt_file() → Plain-Text String

For .pdf:

load_pdf_file() → Markdown String

For any unsupported extension:

Raise Unsupported-File Error

After extraction, both supported formats enter the same existing ingestion pipeline.

The complete routing design is:

File Path

- Detect Extension
- Select Loader
- Extract Text
- Generate Note ID and Source
- Duplicate Check
- ingest_note()
- Chunking
- Embeddings
- Vector Storage

This keeps file extraction separate from the existing storage logic.

The loader is responsible for converting a file into text.

The ingestion layer is responsible for preparing and storing that text as searchable knowledge.

PDF Ingestion

Why PDF Ingestion Requires a Parser

A PDF is not equivalent to a plain-text file with a different extension.

PDF documents may contain:

- Headings

- Paragraphs
- Multiple columns
- Bullet lists
- Tables
- Equations
- Hyperlinks
- Page headers and footers
- Embedded images
- Scanned content
- Complex visual layouts

A basic text extractor may retrieve the words while losing the relationships between them.

For example, a table could be reduced to a sequence of disconnected words and numbers with no clear connection between rows and columns.

Because the Personal AI Agent will eventually work with lecture notes, study documents, and structured PDFs, the file-ingestion layer needs to preserve document structure when possible.

Docling Installation

Docling was installed inside the project's Python virtual environment.

Docling is used as the primary structured PDF parser.

Its role is:

PDF → Structured Docling Document → Markdown String

On the first PDF conversion, Docling downloaded its required document-processing models.

The first attempt was interrupted by a connection failure while downloading model files from Hugging Face.

The error was caused by the remote connection closing during the download rather than by the project code.

When the test was run again, the missing model files downloaded successfully.

The models are now cached locally, allowing later conversions to run without repeating the complete first-time download.

PDF File Loading

PDF loading was added to:

backend/app/ingestion/file_loader.py

A function was created:

load_pdf_file(file_path)

The function uses Docling's:

DocumentConverter

The workflow is:

PDF Path

→ DocumentConverter

→ ConversionResult

→ DoclingDocument

→ export_to_markdown()

→ Markdown String

The result returned directly by Docling is not initially a string.

converter.convert(file_path)

returns a ConversionResult object.

The important structured document is available through:

result.document

This is a Docling document object containing the reconstructed document content and structure.

The structured document is converted into a normal Python string using:

result.document.export_to_markdown()

The Markdown string can then be chunked, embedded, stored, retrieved, and passed to the language model.

Why Markdown Is Used

Markdown is structured plain text.

It uses simple text symbols to preserve relationships such as:

- Headings
- Lists
- Tables
- Links
- Equations

Example heading:

Neurogum: A focus booster

Example bullet:

- Neurogum is a supplement.

Example table:

ID	Name	Product	Amount
1	Joe	Neuro Energy and Focus	39.99
2	Sharma	Neuro Sleep	19.99

The Markdown representation remains a normal string, but it provides structural clues to the LLM.

For example, the vertical separators in a Markdown table allow the model to understand which value belongs to which row and column.

The PDF ingestion workflow therefore becomes:

PDF → Docling → Structured Markdown → Chunking → Embeddings → Vector Storage

PDF Extraction Testing

Headings

A Word heading was added to a PDF and successfully extracted as Markdown.

Example:

Neurogum: A focus booster

This confirms that Docling can recognize a styled heading and preserve it as structured text.

An important distinction was identified between:

- A document heading
- A repeating Word page header

Text placed inside Microsoft Word's Header area was omitted from the Markdown output.

This is expected behavior because Docling treats repeating page headers and footers as page furniture rather than primary document content.

For meaningful document titles and section headings, Word heading styles should be used inside the document body.

Paragraphs

Normal body paragraphs were extracted successfully.

Paragraph order and separation were preserved.

The resulting output remained readable and suitable for:

- Chunking
- Embedding
- Semantic retrieval
- LLM prompting

This test passed.

Bullet Lists

A bulleted list was added to to the test PDF.

Docling converted it into Markdown list items.

Example:

- Neurogum is a supplement.
- It is chewing gum or a mint.
- It comes in blue boxes.

The structure clearly indicates that the lines belong to the same list.

This test passed.

Standard Tables

A normal single-page table was tested.

The original table contained:

- Column headings
- Multiple rows
- Text values
- Numeric values

Docling preserved the table using Markdown.

Example:

ID	Name	Product	Amount
1	Joe	Neuro Energy and Focus	39.99
2	Sharma	Neuro Sleep	19.99

The output correctly preserved:

- Column names
- Row order
- Cell relationships
- Text values

- Numeric values

This structure is suitable for an LLM because the relationships between each value and its column remain visible.

This test passed.

Empty Table Cells

A table containing some missing values was also tested.

When the table remained together on one page and retained a clear overall structure, Docling preserved empty cells correctly.

This shows that missing data alone does not necessarily prevent successful table extraction.

However, table reliability decreases when empty cells are combined with:

- Unnamed columns
- Mostly empty rows
- Page breaks
- Isolated values

Cross-Page Tables

An intentionally messy table was created to test Docling's limits.

The table contained:

- Missing values
- An extra unnamed column
- Empty rows
- Isolated values
- A page break between the header and the data

The table header appeared at the bottom of one page.

The data rows appeared on the following page.

Docling failed to connect the two parts reliably.

It produced:

- One header-only Markdown table
- One separate data table without its proper header

Some values were also assigned to the wrong column.

For example, an ID value from the first column was moved into the final unnamed column.

The resulting Markdown was syntactically table-like but semantically incorrect.

This test failed.

The cause is that PDFs generally store page-level text and coordinates. They do not always contain a reliable semantic instruction stating that a table continues onto the next page.

Docling must infer that relationship from visual layout.

The test established:

Docling can sometimes recognize cross-page tables, but cross-page reconstruction cannot be assumed to be reliable.

For documents under the user's control, the preferred practices are:

- Keep a short table on one page.
- Move the complete table to the next page when necessary.
- Repeat column headings on later pages for long tables.
- Avoid leaving only the header row at the bottom of a page.
- Give every meaningful column a name.
- Avoid unnecessary empty columns.

Cross-page tables from external documents may require later ingestion-quality checks.

Equation Testing

Structured Equations

A structured equation linked from Wikipedia was included in the test PDF.

Docling preserved a LaTeX-like representation:

$$E=mc^2$$

The notation:

2

preserves the superscript relationship.

The Markdown link also preserved the associated URL.

This format is suitable for an LLM because the mathematical structure remains explicit.

This test passed.

Plain-Text Equations

An equation written directly as:

E = mc^2

was extracted correctly.

The caret notation clearly indicates that 2 is the exponent.

This format is suitable for the current text-based ingestion pipeline.

This test passed.

Visually Formatted Superscripts

A separate equation was created using ordinary text with a manually formatted superscript.

Visually, it appeared as:

E = mc²

However, the PDF stored the superscript as a smaller, separately positioned text object.

Docling detected the 2, but it did not preserve its connection to the equation.

The output became:

2

followed separately by:

Text equation example: E = mc

This was a layout-relationship failure rather than a character-detection failure.

The test failed.

Formula enrichment was enabled and tested.

However, formula enrichment did not correct the result because Docling had already classified the main expression and superscript as separate ordinary text elements rather than one formula.

For user-created documents, more reliable approaches include:

- Using Word's Equation Editor
- Writing the Unicode ² character
- Writing explicit plain-text notation such as mc^2

For external PDFs, visually formatted superscripts remain a known extraction limitation.

Image Testing

Image Detection

PDFs containing embedded images were tested.

Docling detected the images and inserted the following Markdown placeholder:

```
<!-- image -->
```

This confirms that Docling recognized that an image existed and preserved its approximate location in the document flow.

Image detection passed.

Image Meaning

An apple photograph was placed inside a PDF.

The Markdown output contained:

```
<!-- image -->
```

A manually written caption below the image was preserved:

the above picture is an apple

However, Docling did not independently determine that the image contained an apple.

The LLM would only know that it was an apple because the manually written caption was included as ordinary text.

Without that caption, the vector database and LLM would receive only an image placeholder.

Image understanding was therefore not implemented by the current pipeline.

Text Inside Images

A screenshot containing large, readable text was added to another PDF.

Docling again produced:

```
<!-- image -->
```

The visible words inside the screenshot were not included in the Markdown output.

RapidOCR was active during document processing, but the current Docling export did not reliably extract the screenshot's text.

Forced full-page OCR was considered and tested as a possible approach.

However, forced full-page OCR introduces tradeoffs:

- Slower processing
- Possible duplication of native PDF text
- Possible OCR spelling errors
- Possible interference with otherwise correct tables and layouts

Image OCR and image understanding were therefore deferred to a future sprint.

The current PDF ingestion pipeline should be considered text-focused.

Current PDF Extraction Results

The current Docling-based PDF loader has the following validated behavior:

Content Type	Result
Document headings	Pass
Body paragraphs	Pass
Bullet lists	Pass
Standard single-page tables	Pass
Clear empty table cells	Pass
Hyperlinks	Pass
Structured equations	Pass
Plain-text equations using ^	Pass
Repeating page-header exclusion	Expected behavior
Visually formatted superscripts	Limitation
Complex cross-page tables	Limitation
Image detection	Pass
Image meaning	Not supported
Reliable text extraction from embedded images	Not supported

These limitations do not prevent completion of the core file-ingestion stage.

They identify advanced document-processing problems that can be addressed separately without blocking ordinary TXT and PDF ingestion.

PDF Ingestion Workflow

After successful Markdown extraction, PDF content enters the same ingestion pipeline used for text files.

The completed workflow is:

PDF Path

→ Validate File

- load_pdf_file()
- Docling Conversion
- Markdown String
- Dynamic Note ID
- Dynamic Source
- Duplicate Check
- ingest_note()
- Sentence-Based Chunking
- Chunk IDs and Metadata
- EmbeddingGemma through Official Ollama on Port 11435
- Embedding Vectors
- ChromaDB Storage

Once stored, the PDF knowledge becomes available to the Stage 2 retrieval and answering pipeline.

The question workflow remains:

User Question

- Query Embedding
- ChromaDB Similarity Search
- Retrieved PDF Chunks
- Prompt Builder
- Qwen3 4B through IPEX-LLM on Port 11434
- Grounded Answer

Unified File Ingestion

Stage 3 produced a unified file-ingestion workflow for supported document types.

The final architecture is:

User Supplies File Path

- Determine File Extension
- TXT or PDF Loader
- Extract Structured Text
- Generate Note ID from Filename
- Generate Source from Filename
- Check Whether Note Already Exists
- Send Extracted Text to Existing Ingestion Pipeline
- Create Chunks
- Generate Embeddings
- Store Searchable Records

The ingestion system now accepts real documents instead of requiring manual string input.

Separation of Responsibilities

The Stage 3 design keeps the system modular.

File Loader

Responsible for:

- Opening supported files
- Extracting their contents
- Returning a text string
- Preserving PDF structure through Markdown where possible

File Ingestion Layer

Responsible for:

- Detecting the file type
- Selecting the correct loader
- Creating the note ID

- Creating source metadata
- Preventing duplicate ingestion
- Passing extracted text to ingest_note()

Existing Note Ingestion

Responsible for:

- Chunking extracted text
- Creating chunk IDs
- Creating chunk metadata
- Sending chunks to vector storage

Vector Database Layer

Responsible for:

- Generating embeddings
- Storing chunks and metadata
- Checking for existing note records
- Retrieving semantically related knowledge
- Deleting individual chunks or complete notes when requested

This separation means future file types can be added without rewriting the established RAG pipeline.

Deleting Ingested Knowledge

The system's vector collection supports deleting:

- One specific chunk
- Multiple chunks
- Every chunk belonging to one note

An individual chunk can be deleted using its unique Chroma record ID.

A complete document can be deleted using its metadata note_id.

For example, deleting with:

note_id: Neurogum

removes every chunk associated with the Neurogum.pdf document.

This is useful when:

- A file was ingested accidentally.
- A document needs to be replaced.
- Extracted content was incorrect.
- A newer version of a document should be ingested.

Deletion can be verified by retrieving records with the same note_id and confirming that the returned ID list is empty.

File Ingestion Validation

The complete file-ingestion workflow was tested.

The validated TXT workflow is:

TXT File

- Text Extraction
- Dynamic Note Identification
- Duplicate Check
- Chunking
- Embeddings
- Vector Storage

The validated PDF workflow is:

PDF File

- Docling Conversion
- Markdown Extraction
- Dynamic Note Identification

→ Duplicate Check

→ Chunking

→ Embeddings

→ Vector Storage

The general file router was also tested to confirm that the correct loader is selected using the file extension.

Unsupported extensions are rejected clearly.

Duplicate files are detected before ingestion.

This confirms that the application can now transform real user files into searchable personal knowledge.

Project Structure Update

The relevant Stage 3 files are organized under:

backend/app/ingestion/

This area now contains logic for:

- General file loading
- TXT loading
- PDF loading
- File-type routing
- Note ingestion
- Text chunking
- Ingestion tests

The local documents used for testing are stored under:

backend/data/

Personal documents and persistent vector data remain excluded from Git where appropriate.

The project continues to use standardized imports beginning with:

app.

Git Milestone

The completed Stage 3 work was committed to the development branch.

The development branch was then merged into the main branch using a non-fast-forward merge.

This created an explicit merge commit representing the completed Stage 3 milestone.

The local main branch was verified as:

- Clean
- Fully committed
- Ahead of the remote branch before pushing

The completed main branch was then ready to be pushed to:

origin/main

This preserves Stage 3 as a stable project checkpoint before beginning the next development stage.

Stage 3 Result

Stage 3 successfully achieved its primary goal:

Allow the Personal AI Agent to ingest real TXT and PDF documents into its local knowledge system.

The system can now:

- Accept a file path.
- Detect the file extension.
- Validate the supplied file.
- Read plain-text documents.
- Parse PDF documents using Docling.
- Convert structured PDF content into Markdown.

- Preserve headings, paragraphs, lists, standard tables, links, and many equations.
- Generate note IDs automatically from filenames.
- Store the complete filename as source metadata.
- Detect previously ingested notes.
- Prevent unintended duplicate ingestion.
- Pass extracted content into the existing chunking pipeline.
- Generate embeddings locally.
- Store file-derived knowledge persistently.
- Make ingested documents available to semantic retrieval.
- Use retrieved document knowledge for grounded local LLM answers.

The project has therefore advanced from:

Manual Text Ingestion

to:

General Local File Ingestion

Known Limitations

The following limitations were identified but are not required for completion of Stage 3:

- Visual understanding of embedded images
- Reliable OCR of text inside embedded images
- Automatic image descriptions
- Reliable reconstruction of complex cross-page tables
- Perfect handling of manually formatted superscripts
- Automatic correction of malformed extracted tables
- Extraction-quality scoring
- Automatic replacement of modified documents
- Collision-resistant IDs for files with identical names in different folders

- Table-aware and structure-aware chunking
- Preservation of page numbers and detailed source locations
- Additional file formats such as DOCX, PPTX, HTML, and Markdown

These items can be addressed in future sprints as the ingestion system becomes more advanced.

Stage 3 Status

Stage: Stage 3 — File Ingestion

Sprint: Sprint 3

Goal: TXT and PDF File Ingestion

Status: COMPLETE

The completed system now supports:

TXT/PDF File → Structured Text → Chunking → Embeddings → Vector Storage → Retrieval → Grounded Answer

The Personal AI Agent now has a functional document-ingestion foundation that builds directly on the working RAG pipeline completed during Stage 2.